



Prog II:

Ponteiros e Alocação Dinâmica

Profª. Tainá Isabela



Ponteiros

Os ponteiros são um dos conceitos mais fundamentais e poderosos da linguagem C++, permitindo ao programador manipular diretamente endereços de memória. Em termos simples, um ponteiro é uma variável cujo valor é o endereço de outra variável na memória do computador.

Isso permite manipular diretamente a memória, aceder e modificar valores de variáveis de forma indireta, além de possibilitar a alocação dinâmica de memória e o uso eficiente de arrays e funções.

Ponteiros

Ponteiros compreendem um dos recursos mais poderosos da linguagem C++. Um ponteiro é uma variável especial que armazena o endereço de outra variável (que contém informação).

Em um programa, toda informação manipulada pelo programa, sejam dados ou instruções, reside na memória (do computador) em determinado endereço e ocupa uma quantidade definida de bytes. Assim, quando você executa o programa, essas variáveis residem em endereços específicos

Operadores de Endereço

Em C++, o endereço de uma variável ou objeto pode ser referenciado fazendo uso do operador de endereço **&**.

Lembre-se de que, quando coloca o operador endereço **&** no tipo de dado de um parâmetro (formal) do protótipo de uma função e cabeçalho da função, esse parâmetro recebe o endereço do correspondente argumento quando a função é chamada.

Operadores de Endereço

Todavia, existe pouca utilidade em mostrar os endereços de memória de uma variável ou objeto. Então, por que ponteiro é um recurso poderoso?

Esse poder advém do fato de que uma variável do tipo ponteiro armazena valores de endereço.

Agora, quando se fala de endereços, esses endereços também são armazenados de maneira similar. Portanto, uma variável que armazena um **valor de endereço** é denominada variável ponteiro ou simplesmente ponteiro.

Operadores de Endereço

C++ permite armazenar o endereço de memória em um tipo especial de variável ou objeto, o ponteiro. Assim, quando você define um ponteiro, o tipo de variável ou objeto para o qual ele aponta deve também ser definido.

O tipo de variável ou objeto para o qual o ponteiro aponta é referenciado como sendo o tipo base do ponteiro.

Em outras palavras, esse tipo base do ponteiro determina como a variável ou objeto será interpretado.

```
#include <iostream>
using namespace std;

// Programa para ilustrar uso de ponteiros

int main()
{
    int x = 11; // declara e inicializa a variável x
    int y = 22; // declara e inicializa a variável y
    cout << "\nValor de x = " << x;
    cout << "\nValor de y = " << y;

    int *xPtr; // declara xPtr como variável ponteiro
    int *yPtr; // declara yPtr como variável ponteiro
    xPtr = &x; //atribui o conteúdo de x a xPtr
    yPtr = &y; //atribui o conteúdo de y a yPtr

    cout << "\n\nEndereco de x = " << &x;
    cout << "\nEndereco de y = " << &y;
    cout << "\n\nValor de xPtr = " << xPtr;
    cout << "\nValor de yPtr = " << yPtr;
    cout << "\n\nValor de *xPtr = " << *xPtr;
    cout << "\nValor de *yPtr = " << *yPtr;
    cout << endl << endl;
    system("PAUSE");
    return 0;
}
```

Operações com ponteiros

As linguagens C e C++ oferecem um suporte especial para nomes de arrays. O compilador interpreta o nome de um array como o endereço de seu primeiro elemento. Assim, se `a` é um array, as expressões `&a[0]` e `a` são equivalentes.

Note que, uma vez que tenha o endereço de um dado, você pode obter esse dado. Esse conhecimento de endereço de memória de uma variável ou array permite manipular o conteúdo desses itens usando ponteiros

Declaração e Inicialização

Para declarar um ponteiro, utiliza-se o operador *:

```
int *ptr; // Ponteiro para inteiro  
double *dptr; // Ponteiro para double
```

Para inicializar um ponteiro, atribui-se o endereço de uma variável usando o operador &:

```
int valor = 10;  
int *ptr = &valor; // ptr aponta para valor
```

Aceder a Valores com Ponteiros

O operador * (desreferenciação) permite aceder o valor armazenado no endereço apontado pelo ponteiro:

```
int valor = 20;  
int *ptr = &valor;  
cout << *ptr << endl; // Imprime 20
```

Se alterarmos o valor via ponteiro, a variável original também é alterada:

```
*ptr = 30;  
cout << valor << endl; // Imprime 30
```

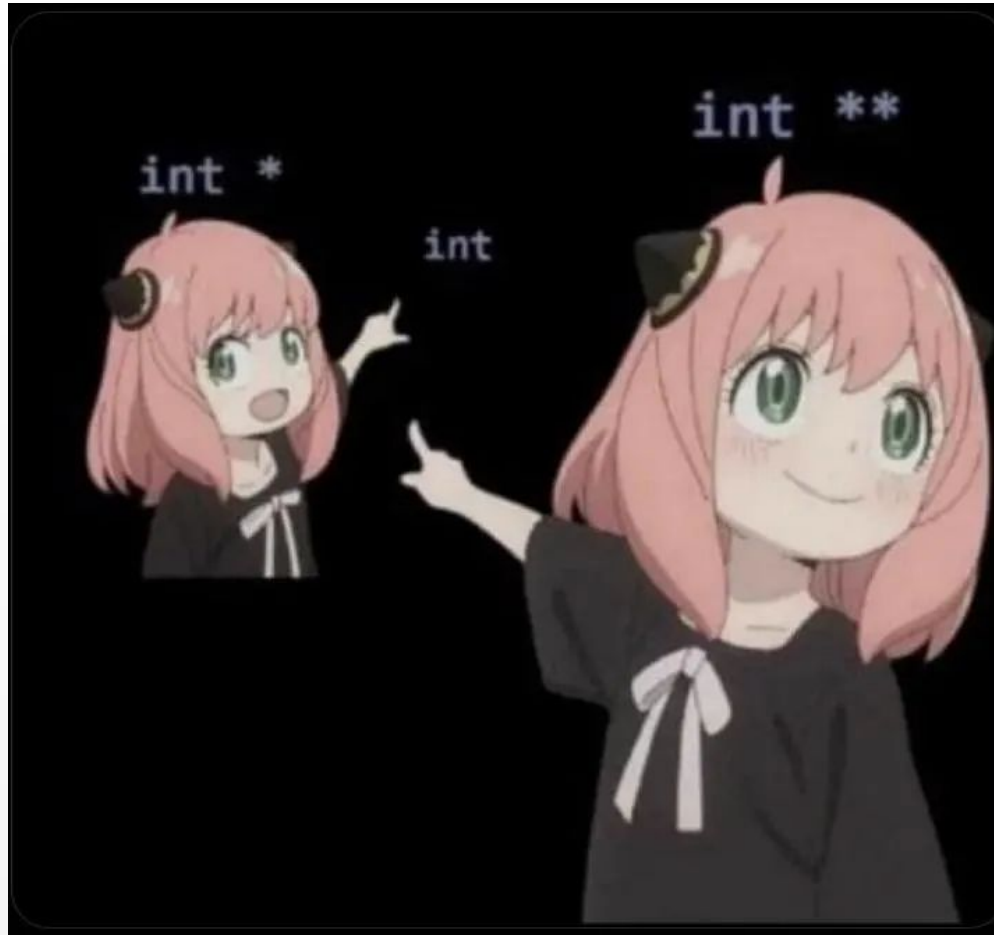
Ponteiros e Arrays

O nome de um array é, na prática, um ponteiro para o seu primeiro elemento:

```
int arr[3] = {1, 2, 3};  
int *ptr = arr;  
cout << ptr[1] << endl; // Imprime 2
```

É possível percorrer arrays usando aritmética de ponteiros:

```
for (int i = 0; i < 3; i++) {  
    cout << *(ptr + i) << endl;  
}
```



Alocação Dinâmica de Memória

Ponteiros são essenciais para alocação dinâmica de memória, usando os operadores **new** e **delete**:

```
int *p = new int; // Aloca um inteiro
*p = 42;
cout << *p << endl; // Imprime 42
delete p; // Liberta a memória
```

Alocação Dinâmica de Memória

Para arrays dinâmicos:

```
int *v = new int[5]; // Aloca array de 5 inteiros
for (int i = 0; i < 5; i++) v[i] = i * 2;
delete[] v; // Liberta o array
```

Passar ponteiros para Funções

Ponteiros permitem passar variáveis por referência para funções, possibilitando a modificação do valor original:

```
void incrementa(int *p) {  
    (*p)++;  
}  
  
int main() {  
    int x = 5;  
    incrementa(&x);  
    cout << x << endl; // Imprime 6  
}
```

Ponteiros para Funções

É possível criar ponteiros que apontam para funções, permitindo selecionar dinamicamente qual função executar:

```
int soma(int a, int b) { return a + b; }
int subtrai(int a, int b) { return a - b; }

int main() {
    int (*operacao)(int, int);
    operacao = soma;
    cout << operacao(2, 3) << endl; // Imprime 5
    operacao = subtrai;
    cout << operacao(5, 2) << endl; // Imprime 3
}
```


Questionário



Exercícios

1. Escreva um programa que declare e inicialize duas variáveis inteiras x e y. Em seguida, você deve exibir o conteúdo dessas variáveis e seus respectivos endereços.
2. Escreva um programa que crie um array de double e que permita entrar com até 100 valores. Entretanto, quando seu programa ler os dados digitados pelo usuário, ele deve usar a instrução:
 - a. `cin >> *(a + j); // usando *(a+j) para armazenar em a[j]`
 - b. e, quando acumular os valores lidos, seu programa deve usar:
 - c. `soma += *(aPtr + j); // usando *aPtr para acessar a[j]` Por fim, seu programa deve exibir a soma e a média dos valores digitados.